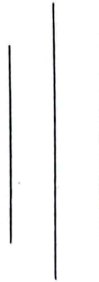




TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 5

Title:- Introduction to Joins & creating views

Submitted by:-

Name:- Sumajan Shrestha

Roll No:- 38

Submitted To:- Himel Chand Thapa

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/08

TITLE : Introduction to Joins & Creating views

OBJECTIVE :

To understand the concept and application of SQL JOIN operations to combine records from multiple tables.

To explore various types of JOINS including the INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

To learn how to create and manage views in SQL, enabling simplified access to data from one or more tables.

To practice inserting, updating and deleting the records from views as well as manipulating underlying tables through views.

THEORY :

SQL joins are powerful tools used to combine rows from two or more tables based on related columns between them. There are different types of joins, each providing distinct ways of retrieving and combining data from related tables.

INNER JOIN :

The INNER JOIN keyword selects records that have matching values in both tables. If no match is found, the rows are excluded from the result set.

Eg :

```
SELECT columns FROM table1 JOIN table2 ON table1.column  
= table2.column;
```

LEFT JOIN:

The LEFT JOIN (or LEFT OUTER JOIN) returns all rows from the left table, and the matched rows from the right table. If there is no match, NULL values will be shown for columns from the right table.

Example:

```
SELECT columns
```

```
FROM table1
```

```
LEFT JOIN table2 ON table1.Column = table2.Column;
```

RIGHT JOIN:

The RIGHT JOIN (or RIGHT OUTER JOIN) is the opposite of the LEFT JOIN. It returns all rows from the right table, and the matched rows from the left table. If there is no match, NULL values will appear for columns from the left table.

Example:

```
SELECT columns
```

```
FROM table1
```

```
RIGHT JOIN table2 ON table1.Column = table2.Column;
```

FULL OUTER JOIN:

The FULL OUTER JOIN returns all rows when there is a match in either table. If there is no match, NULL values will be displayed for non-matching rows.

Example:

```
SELECT columns
```

```
FROM table1
```

```
FULL OUTER JOIN table2 ON table1.Column = table2.Column;
```


views in SQL:

A view is a virtual table based on the result of an SQL query. It provides a way to simplify complex queries by representing them as a single virtual table, allowing easier data retrieval and manipulation. Views can include multiple joins, filters and calculations. They are especially useful for security and abstraction, where users are restricted from accessing specific columns or rows from underlying tables.

Syntax to create a view:

SQL:

```
CREATE VIEW view_name AS  
SELECT columns  
FROM table  
WHERE condition ;
```

Views can also be updated or deleted, and changes in the view may reflect in the underlying tables, depending on how the view is structured.

OBSERVATIONS:

→ In this lab, we worked with several tables including Employee, Teacher, Booklist, Book, student and Issues. We performed different join operations, explored views and manipulated data by inserting and deleting records through views.

LAB TASKS:

→ Perform Join operation on teacher and employee table and display the Ename, faculty and salary.

SQL:

```
SELECT e.ename, t.faculty, e.salary  
FROM teacher t  
JOIN Employee e ON t.eid = e.eid;
```

perform left join on the Booklist and Book table:

SQL:

```
SELECT b1.isbn, b1.name AS BooklistName, b.bid,  
b.bname FROM Booklist b1  
LEFT JOIN Book b ON b1.name = b.bname;
```

perform right join on Booklist and Book table:

SQL:

```
SELECT b1.isbn, b1.name AS BooklistName, b.bid, b.bname  
FROM Booklist b1  
RIGHT JOIN Book b ON b1.name = b.bname;
```

perform full join on student and Issues table:

SQL:

```
SELECT s.ID, s.Name, i.iid, i.name AS IssuedTo  
FROM student s  
FULL OUTER JOIN Issues i ON s.ID = i.iid;
```

Display those employees' names and salaries whose name starts with 'S' and whose name consists of 'ni' as a substring:

SQL:

```
SELECT ename, salary FROM Employee  
WHERE ename LIKE 'S%' AND ename LIKE '%ni%';
```


5. Display the name of the employee who is also a teacher :

SQL :

```
SELECT e.ename  
FROM Employee e  
JOIN teacher t ON e.eid = t.eid;
```

Display all employees' names except those who are teachers :

SQL :

```
SELECT e.ename  
FROM Employee e  
WHERE e.eid NOT IN (SELECT t.eid FROM teacher t);
```

Create a view Employee-view which consists of eid, ename, salary as attributes :

SQL :

```
CREATE VIEW Employee-view AS  
SELECT eid, ename, salary  
FROM Employee;
```

Insert a new record in the recently created view and display the contents of the primary table :

SQL :

```
INSERT INTO Employee-view (eid, ename, salary)  
VALUES (5, 'krishna', 6000);  
SELECT * FROM Employee;
```

10. Delete the information from the view where salary is less than 5000.

SQL:

```
DELETE FROM Employee_view  
WHERE salary < 5000;  
SELECT * FROM Employee;
```

DISCUSSION:

This lab demonstrated various types of SQL JOIN operations, including INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN, each allowing us to combine related data from multiple tables, with unmatched records represented by NULL values. We also explored views, which simplify complex queries by creating virtual tables. In this lab, we created a view, inserted new records and deleted data, showing that views can abstract and manage data effectively. Additionally, the use of LIKE for pattern matching allowed us to efficiently filter employee names based on specific character patterns, enhancing query precision.

CONCLUSION:

In this lab, we implemented various JOIN operations to combine data from multiple tables and learned how to create and manage views to simplify data retrieval and updates. The lab highlighted the importance of JOINS for merging related data and views for enhancing data security and abstraction, both essential for efficient database management.